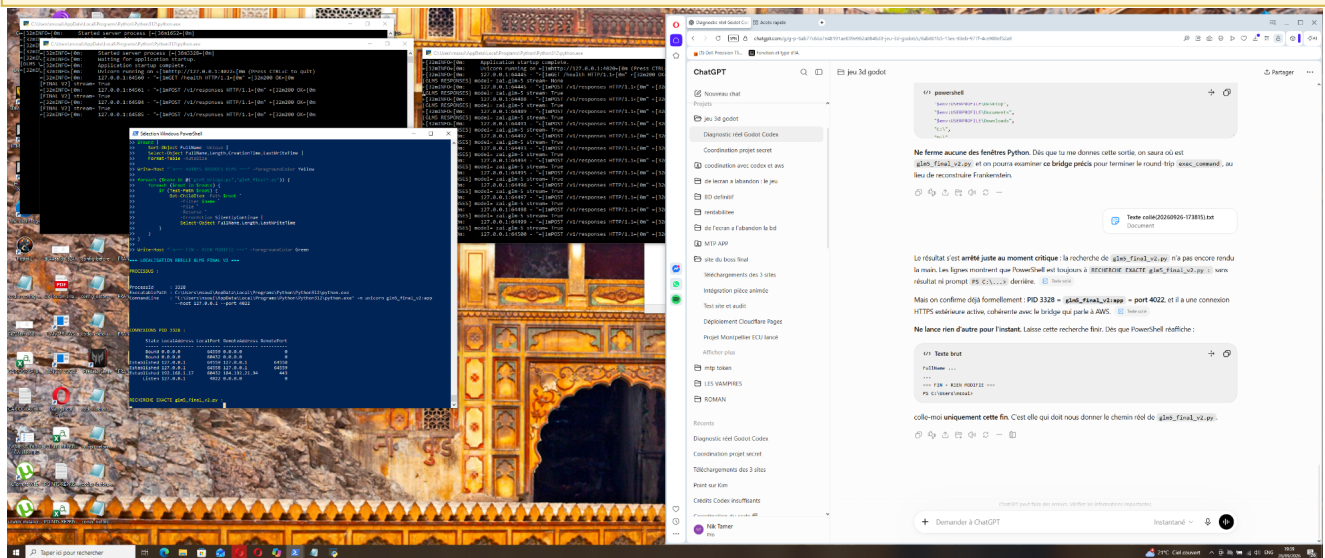


# GLM-5 DANS CODEX WINDOWS

## Chronique illustrée d'un Frankenstein agentique

26 septembre 2026 - Archive technique et visuelle du projet

Cette version illustrée conserve non seulement les décisions techniques, mais aussi les captures d'écran des jalons réels : diagnostic des bridges, connexion à Codex Desktop, découverte d'exec\_command, échec Converse, lancement de la vraie GUI et validation finale de V5.



*Diagnostic initial : plusieurs bridges GLM-5 actifs sont visibles. Le travail repart de l'état réel de la machine, sans recréer Frankenstein.*

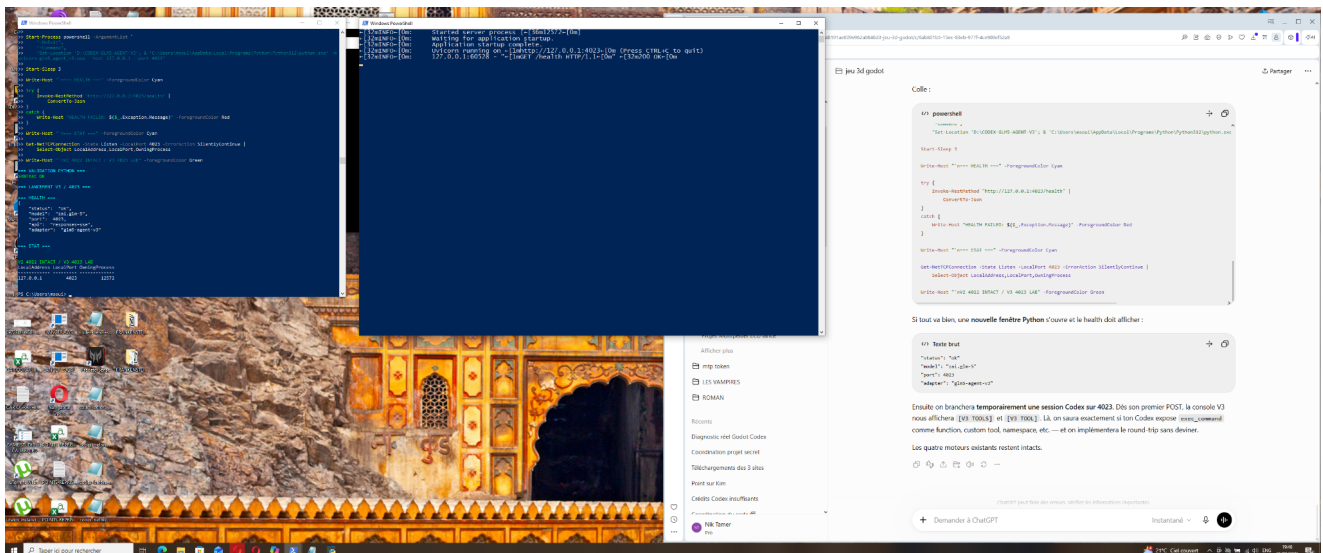
**Portée.** Ce document décrit un jalon technique inhabituel du projet. Il ne prétend pas mesurer combien de personnes dans le monde ont réalisé une intégration comparable.

# 1. Objectif et topologie réelle

L'objectif était de faire de **GLM-5 un ingénieur dans Codex Windows**, tout en conservant **Kimi K3 comme codeur** et **Frankenstein GPT-OSS comme Gold Master**. La règle fondamentale est devenue : ne jamais supposer un chemin ou un port ; inventorier d'abord l'état réel.

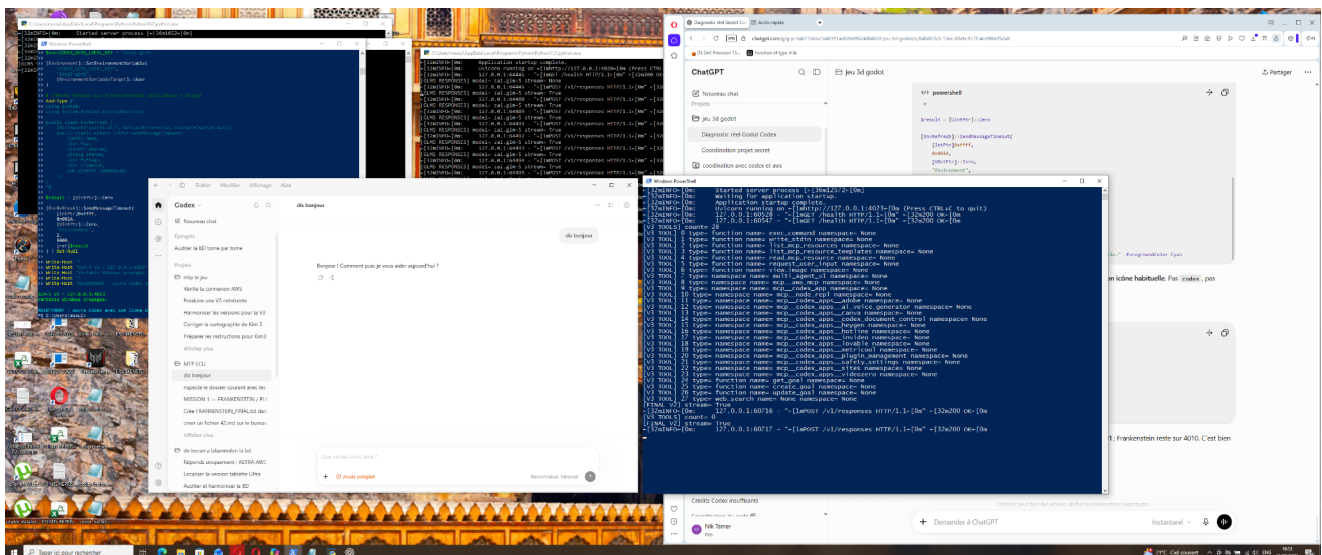
| Port | Rôle / état                                        |
|------|----------------------------------------------------|
| 4001 | Kimi K3 / gateway - protégé                        |
| 4010 | Frankenstein GPT-OSS 120B / Gold Master - protégé  |
| 4021 | Ancien bridge GLM-5                                |
| 4022 | GLM-5 Final V2 - Responses + SSE                   |
| 4023 | V3 - instrumentation des tools Codex               |
| 4024 | V4.x - laboratoire Converse/toolUse/toolResult     |
| 4025 | V5 - Chat Completions natif + tools - base validée |

# 2. V3 : Codex révèle ses outils



Naissance du laboratoire V3 : GLM-5 répond sur le port 4023 et le health check est valide.

Après validation de Responses API et du streaming SSE, V3 est lancé sur **4023**. Son rôle : observer exactement les outils que Codex Desktop envoie au modèle.

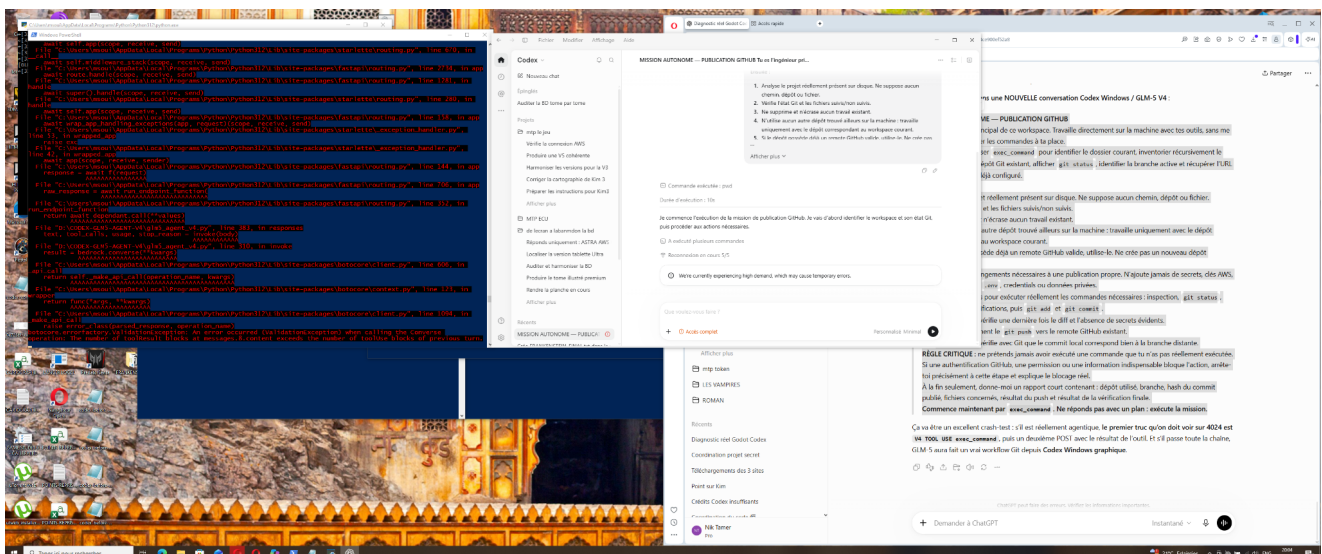


Codex Desktop envoie ses outils au bridge : `exec_command` apparaît comme une fonction native. C'est une découverte déterminante.

La console révèle 28 outils. Le plus important est immédiatement visible : `exec_command` est une fonction normale. Ce résultat permet d'abandonner plusieurs hypothèses héritées du Frankenstein GPT-OSS.

### 3. V4 : le premier aller agentique et le mur Converse

La première architecture complète utilise Bedrock Converse : `toolUse` côté GLM-5, conversion en `function_call` pour Codex, exécution locale, puis reconstruction d'un `toolResult` pour Bedrock.



Premier aller agentique réel : Codex Windows exécute une commande demandée par GLM-5. Le retour vers Converse échoue ensuite sur `toolResult/toolUse`.

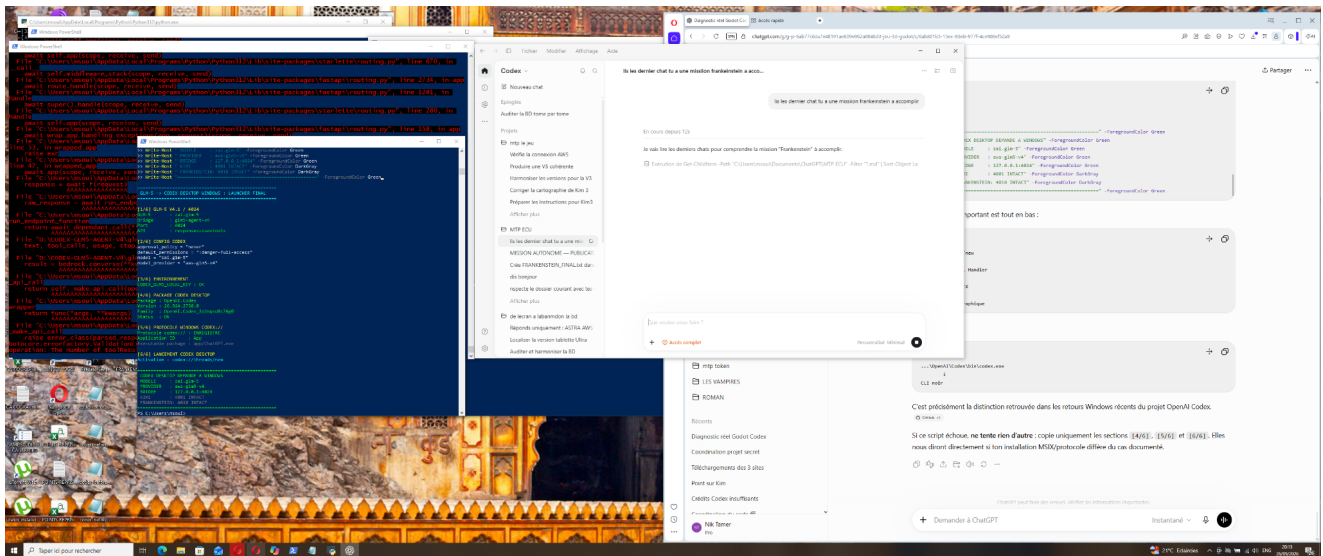
La capture montre un jalon essentiel : **Codex Windows exécute réellement une commande demandée par GLM-5**. L'aller fonctionne. Le crash se produit au retour, lorsque Converse rejette l'historique reconstruit.

Erreur observée : le nombre de `toolResult` dépasse les `toolUse` du tour précédent. L'analyse montre qu'un message assistant texte s'insère entre le `toolUse` et son résultat.

```
user(text)
assistant(toolUse...)
assistant(text) ← rupture
user(toolResult...)
```

## 4. Le piège Codex Windows : CLI contre vraie GUI

Le fichier `codex.exe` trouvé localement ouvrait la TUI noire. L'application graphique réelle est le package Windows **OpenAI.Codex**, qui expose le protocole `codex://`. Le lancement fiable devient `codex://threads/new`.



La vraie interface Codex Desktop est enfin lancée via le protocole Windows `codex://`, et GLM-5 déclenche des commandes locales.

Cette étape permet de conserver Kimi dans son terminal tout en donnant GLM-5 à la vraie interface Codex Desktop.

## 5. Le test natif qui change l'architecture

Plutôt que de continuer à patcher Converse, GLM-5 est testé directement via `bedrock-runtime / invoke_model`.  
Premier résultat : réponse OpenAI Chat Completions valide avec **GLM5\_OK**.

```
{"object": "chat.completion", "model": "zai.glm-5", "choices": [{"message": {"content": "GLM5_OK"}}]}
```

Second test : déclaration d'un outil OpenAI-compatible `exec_command`. GLM-5 produit nativement :

```
finish_reason = "tool_calls"
name = "exec_command"
arguments = {"cmd": "Get-ChildItem -Name"}
call_id = "chatcmpl-tool-..."
```

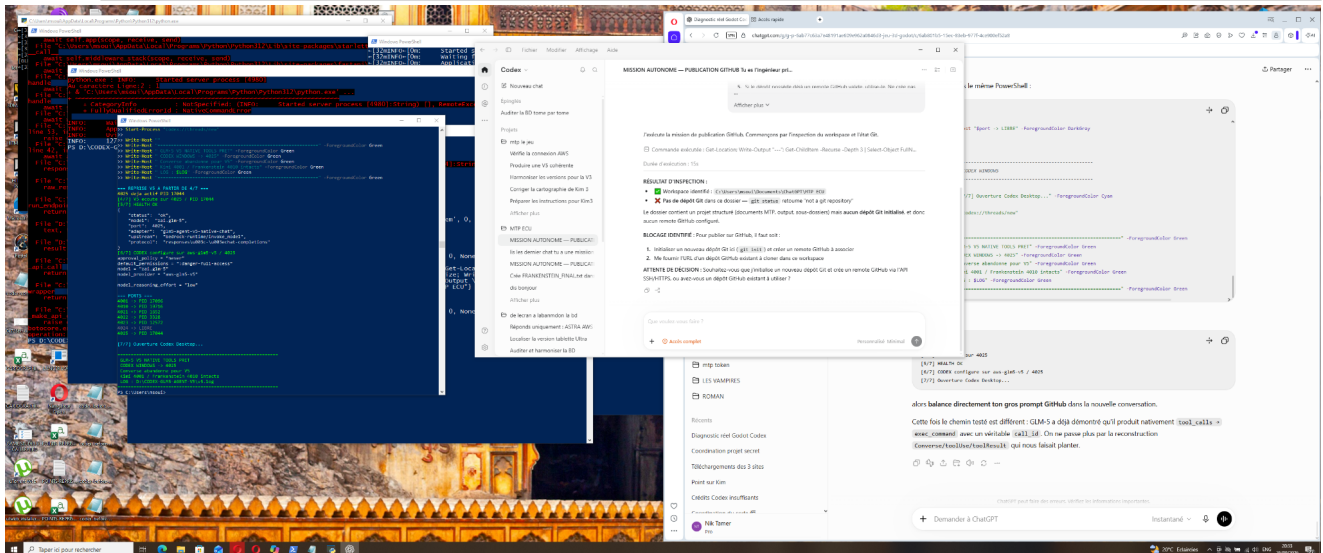
C'est la bascule conceptuelle : plus de Converse pour V5. Le bridge ne traduit plus qu'entre Responses côté Codex et Chat Completions côté GLM-5.

# 6. V5 / 4025 : boucle agentique stable

V5 est créé sur un nouveau port sans détruire les versions précédentes. Le chemin devient :

Codex Responses function\_call  
↔ GLM-5 Chat Completions tool\_calls

Codex function\_call\_output  
↔ GLM-5 role=tool + tool\_call\_id



V5 / port 4025 : boucle agentique stable. GLM-5 inspecte le workspace, exécute des commandes et interprète les résultats sans rupture du stream.

La capture finale montre GLM-5 dans Codex Desktop en train d'accomplir une mission réelle : inspection du workspace, commandes locales, analyse de l'état Git et retour raisonné. Aucun crash Converse, aucune rupture SSE.

# 7. Ce qui est effectivement validé

**Transport** : Codex Desktop ↔ bridge local ↔ AWS Bedrock ↔ GLM-5.

**Outils** : GLM-5 produit des tool\_calls natifs et Codex exécute exec\_command.

**Retour** : le résultat local revient au modèle, qui poursuit son raisonnement.

**Isolation** : Kimi, Frankenstein GPT-OSS et les anciens bridges restent intacts.

**GUI** : la vraie application Codex Windows est lancée via codex://.

# 8. Architecture finale

| Composant       | Fonction                                    |
|-----------------|---------------------------------------------|
| Utilisateur     | Propriétaire / producteur / décision finale |
| GLM-5 V5 / 4025 | Ingénieur agentique dans Codex Windows      |
| Kimi K3 / 4001  | Codeur spécialisé                           |
| GPT-OSS / 4010  | Gold Master / référence / fallback          |
| Codex Desktop   | Interface et exécution des outils locaux    |
| AWS Bedrock     | Inférence GLM-5                             |

# 9. Enseignements

**Inventorier avant de coder.** Les chemins imaginés ont coûté du temps.

**Ne pas confondre transport texte et agenticité.** SSE valide ne garantit pas le round-trip outil.

**Les call\_id sont critiques.** Ils relient décision, exécution et résultat.

**Changer d'architecture au bon moment.** Le test natif Chat Completions a supprimé une couche entière de complexité.

**Conserver les versions.** Chaque Frankenstein intermédiaire est devenu une trace utile de l'évolution.

## 10. Jalon du 26 septembre 2026

**Cette session marque le passage d'une expérimentation de compatibilité à une chaîne agentique réellement opérationnelle dans Codex Desktop Windows. Le modèle ne se contente plus de répondre : il peut demander une action, la faire exécuter localement, recevoir le résultat et continuer son travail.**

**État archivé : GLM-5 V5 / port 4025 = base agentique validée.**

Les captures de ce document proviennent directement de la session de travail et servent de preuves visuelles des principaux jalons.